



АЛГОРИТМЫ И ПРОИЗВОДИТЕЛЬНОСТЬ

ЛЕКЦИЯ 1. Вычислительная сложность алгоритмов и операций над дискретными структурами

Пестунов Андрей Игоревич

Контакты для переписки:

andrey.pestunov@yandex.ru

<https://vk.ru/id572960176>

ПОНЯТИЕ АЛГОРИТМА

Алгоритм – формально описанная вычислительная процедура, получающая исходные данные (**вход**, аргументы, входные данные, input) и выдающая результат вычислений (**выход**, выходные данные, output)

Формальное описание – представление вычислительной процедуры с помощью строгой последовательности **шагов** (действий, элементарных операций, что исключает неоднозначность)

Вычислительная процедура рассматривается в **широком** (не обязательно в арифметическом) смысле. Например, задача сортировки порождает большой спектр алгоритмов, но не является вычислительной в арифметическом смысле

Примеры задач, решение которых предполагает разработку алгоритма:

- **Поиск** элемента в дискретной структуре данных
- Определение **кратчайшего** пути в графе
- **Добавление** нового элемента в дискретную структуру
- Поиск **повторяющихся** элементов в строке

ДЕТЕРМИНИРОВАННЫЕ И ВЕРОЯТНОСТНЫЕ АЛГОРИТМЫ

Детерминированный алгоритм – алгоритм, выполнение которого при одинаковых входных данных всегда приводит к **одинаковому** результату

Правильный алгоритм – алгоритм, результат выполнения которого на всех допустимых данных **удовлетворяет** условию задачи

Вероятностный алгоритм (тип 1) – алгоритм, результат выполнения которого с ненулевой вероятностью может **не удовлетворять** условию задачи (выдает правильный результат с вероятностью ниже 100%)

Вероятностный алгоритм (тип 2) – алгоритм, использующий **рандомизацию** на некоторых шагах

Примеры вероятностных алгоритмов:

- **Обезьяний** метод сортировки
- Тест **Ферма** на простоту
- Рандомизированная **быстрая** сортировка

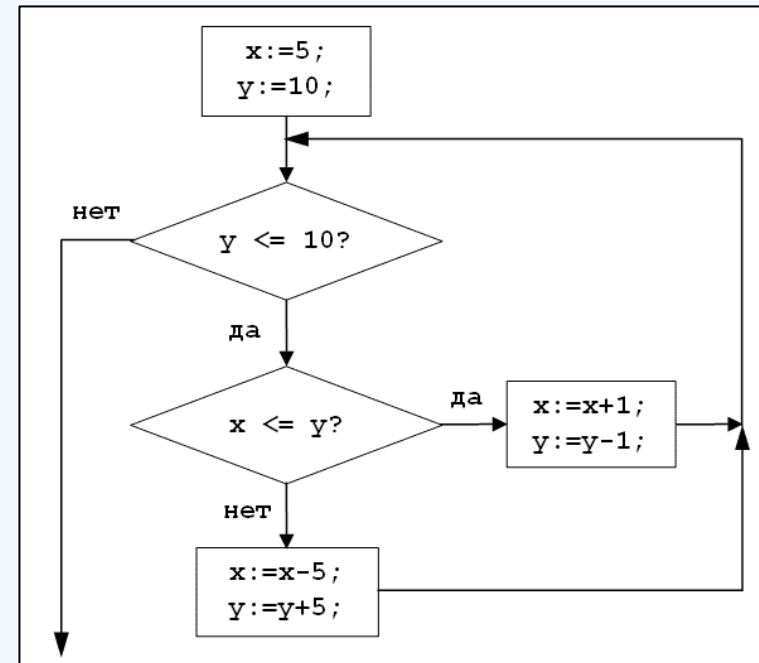
ПРЕДСТАВЛЕНИЕ АЛГОРИТМОВ

Псевдокод – представление алгоритма, на языке, который **напоминает** язык программирования, но является менее формализованным

Блок-схема – представление алгоритма в виде **диаграммы**

HEAPIFY(A, i)

```
1   $l \leftarrow \text{LEFT}(i)$ 
2   $r \leftarrow \text{RIGHT}(i)$ 
3  if  $l \leq \text{heap-size}[A]$  и  $A[l] > A[i]$ 
4      then  $\text{largest} \leftarrow l$ 
5      else  $\text{largest} \leftarrow i$ 
6  if  $r \leq \text{heap-size}[A]$  и  $A[r] > A[\text{largest}]$ 
7      then  $\text{largest} \leftarrow r$ 
8  if  $\text{largest} \neq i$ 
9      then обменять  $A[i] \leftrightarrow A[\text{largest}]$ 
10     HEAPIFY( $A, \text{largest}$ )
```



АЛГОРИТМЫ И БЫСТРОДЕЙСТВИЕ ПРОГРАММ

- При «**небольших**» входных данных и **малом** количестве вызовов пользователя как правило интересует только **результат** работы программы, реализующей алгоритм
- В этих условиях **отличия** быстрых алгоритмов от медленных **незаметны**
- Например, однократный «медленный» **последовательный** поиск в неупорядоченном массиве размера даже, скажем, 1 млн. с точки зрения практической приемлемости выполнится **так же быстро**, как и **двоичный** поиск в упорядоченном массиве
- Если же поисковых запросов будет **много** 1000, 10000 или 100000, то плохая производительность последовательного поиска станет **заметна**, программа будет «тормозить»
- Аналогично неэффективные по **памяти** алгоритмы приведут к **переполнению** ОЗУ

ВЫЧИСЛИТЕЛЬНАЯ СЛОЖНОСТЬ АЛГОРИТМОВ

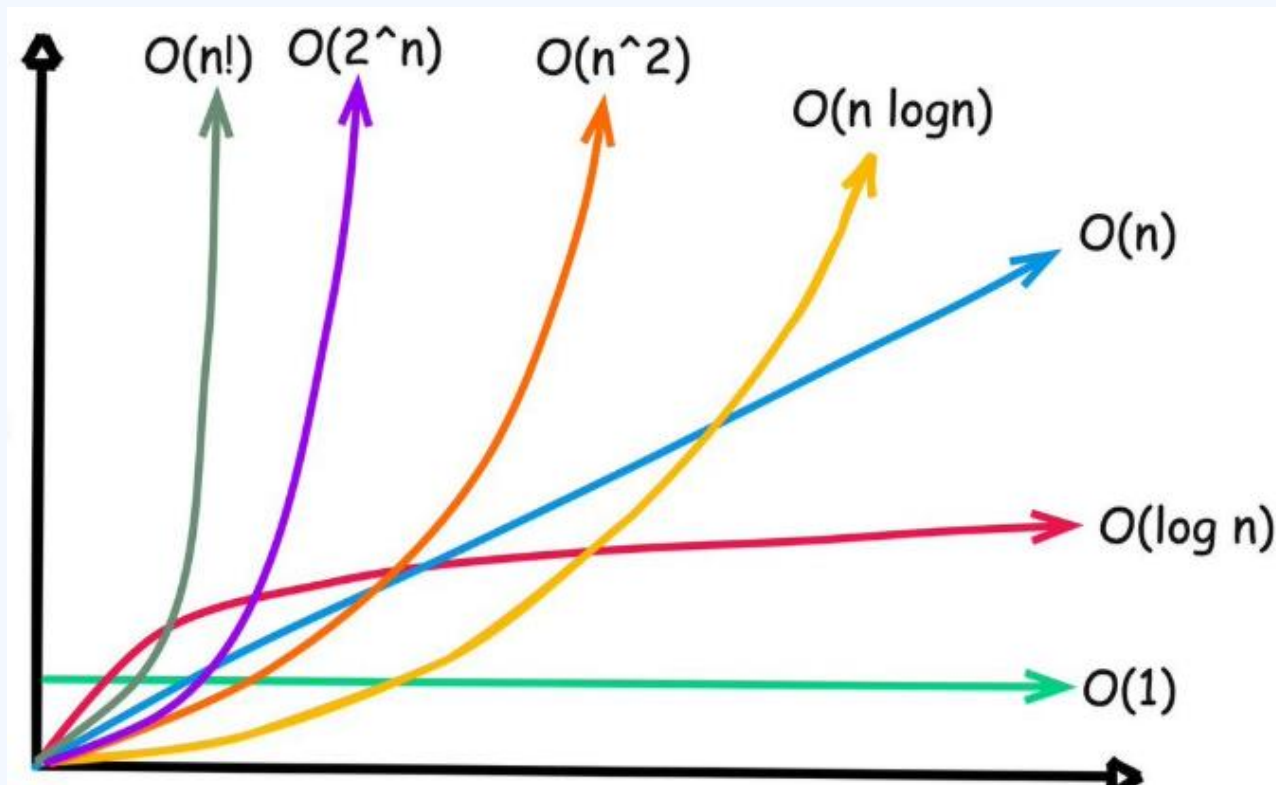
- **Вычислительная сложность** (неформально) – суммарное количество **элементарных** операций при выполнении алгоритма в зависимости от размера и характера входных данных
- **Цель оценки сложности** – **предсказать** затраты памяти и времени выполнения программной реализации в различных случаях
- **Количество элементарных операций** не зависит от программно-аппаратной реализации в отличие от времени исполнения программы
- **Высокая вычислительная сложность** алгоритмов имеет значение при **больших** входных данных; при малых размерах современные ЭВМ эффективно исполняют даже неэффективные алгоритмы
- Поэтому при оценке сложности прежде всего нужен **порядок функции**, а не ее точное выражение; младшие члены функции и коэффициенты оценки сложности можно **отбрасывать**

$$3*N^3 + N_2 + 1 = O(N^3)$$
$$10*N + \text{Log}_2(N) \rightarrow O(N)$$

$$2^N + N^7 \rightarrow O(2^N)$$
$$6*N! + 2^N \rightarrow O(N!)$$

ВИДЫ ВЫЧИСЛИТЕЛЬНОЙ СЛОЖНОСТИ

- **Вид сложности** алгоритма определяется **видом функции**, характеризующей зависимость количества элементарных операций в алгоритме от размера задачи
- **Основными видами** являются логарифмическая, линейная, квадратичная, полиномиальная, экспоненциальная



ЭЛЕМЕНТАРНЫЕ ОПЕРАЦИИ И ПАМЯТЬ

Элементарные операции зависят от **конкретной** задачи

Примеры элементарных операций:

- Сравнение
- Умножение
- Перестановка
- Присваивание

Сортировка: сравнение, присваивание, перестановка

Поиск: сравнение

Алгоритмы **возведения в степень** (в криптографии): умножение

Операции с дискретными **структурами данных:** присваивание

- Помимо элементарных операций вычислительная сложность характеризуется объемом требуемой **оперативной памяти**
- Возможна **балансировка** между количеством элементарных **операций**, требуемой **памятью** и **вероятностью** успеха алгоритма

ПРИМЕР АЛГОРИТМА (ПОЛНЫЙ ПЕРЕБОР)

Базовый алгоритм – **полный перебор**

Используется при решении следующих задач:

- Проверка числа на **простоту**
- **Последовательный** поиск в неупорядоченном массиве
- Вычисление **НОД** и **НОК**
- Вычисление мультипликативной **инверсии** по модулю
- ...

Для многих задач – **не самый эффективный** алгоритм

Вычислительная сложность – **$O(N)$**

Память – **$O(1)$**

- Основная **элементарная операция** – сравнение
- Количество других операций (взятие остатка от деления, умножение) **пропорционально** количеству сравнений
- Лучший случай – **1**, худший – **N**

ПОСЛЕДОВАТЕЛЬНЫЙ ПОИСК КАК ПОЛНЫЙ ПЕРЕБОР

Для всех i от 0 до $N-1$

Если **Mas[i]=x** То Ответ «Есть»
Ответ «Нет»

```
bool indexOf(int x) {  
    for (int i=0; i<N; i++) {  
        if (mas[i]==x) {  
            return i;  
        }  
    }  
    return -1;  
}
```

- **сравнение** –
элементарная
операция
- Сложность – **$O(N)$**
- Лучший случай –
сразу совпадение
- Худший – **нет**
совпадений

ВЕРОЯТНОСТНАЯ ОЦЕНКА СЛОЖНОСТИ

- Пусть все значения в поисковых запросах **равновероятны**, и каждый запрос завершается успехом (выход – позиция найденного элемента от 1 до N).
- Тогда для любого запроса вероятность получить в ответ (обозначим его через t) любой конкретный индекс **одинакова** и равна $1/N$
- Формально – $P(t = i) = 1/N$
- Обозначим **$v(i)$** – сложность (количество сравнений) для поиска i -го элемента, тогда $v(0) = 1, v(1) = 2, v(2) = 3 \dots v(N-1) = N$
- Отсюда $P(v(t) = V) = 1/N$, где V - любое конкретное значение.
- В качестве **средней оценки** для сложности можно взять **математическое ожидание** случайной величины $v(t)$
- $Ev(t) = 1*(1/N) + 2*(1/N) + \dots + N*(1/N) = N*(N-1)/(2*N) \sim N/2 = O(N)$

При разработке алгоритма в общем случае нет информации о точной **последовательности** запросов, поэтому для анализа можно построить (вероятностную) **модель**, характеризующую распределение этих запросов

ДВОИЧНЫЙ ПОИСК

```
left:=0, right:=N-1;
```

```
Пока (left<=right)
  sredn = (left+right)/2;
  Если (x = mas[sredn]) То
    Ответ «Есть»;
  Если (x>mas[sredn]) {
    right := sredn-1;
  }
  Иначе
```

```
left := sredn+1;
```

```
Ответ «Нет»;
```

Порядок сложности –
 $O(\log_2(N))$

СРАВНЕНИЕ СЛОЖНОСТИ ПОСЛЕДОВАТЕЛЬНОГО И ДВОИЧНОГО ПОИСКА

N	100	10^4	10^6	10^8	10^{10}	10^{12}	10^{14}	10^{16}	10^{18}
N/2	100	10^4	10^6	10^8	10^{10}	10^{12}	10^{14}	10^{16}	10^{18}
$\log_2(N)$	~7	~13	~20	~25	~33	~40	~47	~53	~60

ВОПРОС. Насколько корректно, целесообразно и показательно сравнивать сложность этих двух алгоритмов таким прямолинейным способом?

Особенность. Очень **разные** требования к **исходному** массиву.

- Упорядоченный
- Неупорядоченный

Необходимо учитывать **контекст** и смотреть на задачу **шире**

ОЦЕНКА СЛОЖНОСТИ С УЧЕТОМ КОНТЕКСТА

- При оценке сложности важным является не только сложность алгоритма, но и **контекст его использования**.
- **Количество запросов** на исполнение алгоритма, характер входных данных и другая **априорная** информация могут повлиять на общую сложность всей системы алгоритмов.
- Так, хотя высокая эффективность двоичного поиска очевидна, следует понимать, что он применим **только при условии упорядоченности массива**.
- Например, при **малом** количестве запросов на поиск предварительная сортировка **нецелесообразна**. Однако при большом количестве запросов имеет смысл «**вложиться**» в предварительную сортировку

ПОДДЕРЖКА УПОРЯДОЧЕННОСТИ ВМЕСТО СОРТИРОВКИ

- Аналогично можно ввести **еще один параметр** - количество операций вставки/удаления
- Для **поддержания** упорядоченности элементы следует вставлять не произвольно, а в **соответствующее** порядку место
- Реализовать такую операцию для массива возможно при помощи **сдвига** элементов с целью освобождения места при добавлении и с целью избавления от «дыр» при удалении
- При **малом** количестве этих ресурсоемких операций в сравнении с количеством поисковых запросов их осуществление **целесообразно**
- Однако **частое** их исполнение **нивелирует** преимущества двоичного поиска

ОПРЕДЕЛЕНИЕ ПРАКТИЧЕСКОЙ СЛОЖНОСТИ

- Выше проведены **теоретические оценки** сложности последовательного и двоичного поиска. Как показано, сложность зависит от входных данных и может **варьироваться от 1 до N сравнений**.
- Важной задачей является определение **практической** (фактической) сложности - **фактического** количества выполненных операций сравнения.
- Для этого создается специальная **переменная-счетчик**, инициализируется нулем и при каждом сравнении увеличивается на 1.
- В итоге, **после окончания работы** программы эта переменная-счетчик **накопит** в себе количество фактически выполненных сравнений.

Если модель, которая выбрана при расчете теоретической сложности **адекватно** отражает характер входных данных (например, вероятностное распределение запросов, то показатели практической сложности будут **соответствовать** теоретическим расчетам

ПРАКТИЧЕСКАЯ СЛОЖНОСТЬ ПОСЛЕДОВАТЕЛЬНОГО ПОИСКА

```
int indexOf(int x) {  
    int comps = 0;  
    for (int i=0; i<n; i++) {  
        comps++;  
        if (mas[i]==x) {  
            return i;  
        }  
    }  
    cout << srav << endl;  
    return -1;  
}
```

Для сбора **показательной** статистики следует выполнить **достаточное** количество поисковых запросов

ПРАКТИЧЕСКАЯ СЛОЖНОСТЬ ДВОИЧНОГО ПОИСКА

```
int indexOf(int x) {  
    int comps = 0;  
    int left = 0, right = n-1;  
    while (left <= right) {  
        int sredn = (left+right)/2;  
        comps++;  
        if (x == mas[sredn]) {  
            cout << srav << endl;  
            return true;  
        }  
        comps++;  
        if (x > mas[sredn]) {  
            right = sredn-1;  
        } else {  
            left = sredn+1;  
        }  
    }  
    cout << comps << endl;  
    return -1;  
}
```

ЭКСПЕРИМЕНТАЛЬНОЕ ОПРЕДЕЛЕНИЕ ЗАВИСИМОСТИ СЛОЖНОСТИ ОТ РАЗМЕРА МАССИВА

Это **простейший** поверхностный **пример** проведения эксперимента

```
for (int i=50; i<=1000; i+=50) {  
    zapolit();  
    indexOf(rand());  
    cout << comps << endl;  
}  
return 0;  
}
```

МИНИМАЛЬНОЕ КОЛИЧЕСТВО ЗАПРОСОВ ДЛЯ ЭФФЕКТИВНОСТИ ДВОИЧНОГО ПОИСКА

Последовательный поиск - N (сравнений)

Двоичный поиск - $\text{Log}_2(N)$

Квадратичная сортировка - N^2

Быстрая сортировка - $N * \text{Log}_2(N)$

Требуется выполнить K поисковых запросов. При использовании двоичного поиска массив нужно **однократно** отсортировать

Сложность K запросов при **последовательном** поиске составляет $K * N$

При **двоичном** поиске и однократном **квадратичном** методе сортировки – $N * N + K * \text{Log}_2(N)$

При **двоичном** поиске и однократном **быстром** методе сортировки – $N * \text{Log}_2(N) + K * \text{Log}_2(N)$

РАСЧЕТЫ ЗНАЧЕНИЯ K ПРИ КВАДРАТИЧНОЙ СОРТИРОВКЕ

Задача 1. При каком K двоичный поиск с однократной квадратичной сортировкой становится эффективнее последовательного поиска?

$$K * N \geq N * N + K * \text{Log}_2(N)$$

$$K = N * N / (N - \text{Log}_2(N))$$

При больших N выполняется $N \gg \text{Log}_2(N)$,
поэтому $N - \text{Log}_2(N) \approx N$, значит, искомое K приблизительно равно N

Например, если массив имеет размер 1000, то, **начиная с 1000 запросов** двоичный поиск с предварительной однократной квадратичной сортировкой становится эффективнее последовательного поиска.

На следующем слайде посмотрим, что получится,
если использовать **быстрый** метод сортировки

РАСЧЕТЫ ЗНАЧЕНИЯ K ПРИ БЫСТРОЙ СОРТИРОВКЕ

Задача 2. При каком **K** двоичный поиск с однократной быстрой сортировкой становится эффективнее последовательного поиска?

$$K * N = N * \text{Log}_2(N) + K * \text{Log}_2(N)$$

$$K = N * \text{Log}_2(N) / (N - \text{Log}_2(N))$$

При больших **N** значение $\text{Log}_2(N)$ много меньше, чем **N**, поэтому искомое **K** приблизительно равно $\text{Log}_2(N)$.

Например, если массив имеет размер **1000**, то, **начиная с 10 запросов** двоичный поиск с предварительной быстрой сортировкой становится **эффективнее** последовательного поиска. А, скажем, для **1 млн.** элементов двоичный поиск будет эффективнее, **начиная с 20 запросов**.

Отсюда видно, что если запросов **немного**, то двоичный поиск с предварительной сортировкой невыгоден

ДВА ТИПА АЛГОРИТМОВ: ТРИВИАЛЬНЫЕ И НЕТРИВИАЛЬНЫЕ

Компьютерный алгоритм - это набор взаимосвязанных команд (шагов), результатом выполнения которых является решение определенной задачи.

Алгоритм принимает **входные** данные и возвращает **выходные** данные.

Помимо сложности все алгоритмы можно условно разделить и по другому критерию: **наивные (тривиальные)** и **нетривиальные**. Особенностью наивных алгоритмов является то, что они следуют непосредственно из постановки задачи или определения реализуемого объекта.

Наиболее известным наивным алгоритмом является **полный перебор**.

ПОЛНЫЙ ПЕРЕБОР

- Решаемая задача
- Входные данные
- Перебираемое множество
- Проверяемое условие
- Выходные данные

Входными данными является **множество возможных решений** задачи и параметры проверяемого **условия**, а выходными данными - **элемент множества**, для которого выполняется проверяемое условие

ЗАДАЧА ПОИСКА

Входные данные	Перебираемое множество	Проверяемое условие	Выходные данные
- Массив , в котором выполняется поиск - Искомый элемент	Все элементы массива	Очередной перебираемый элемент равен искомому	- Индекс найденного элемента или -1, если элемент не найден
- Множество , в котором выполняется поиск - Искомый элемент	Все элементы множества	Очередной перебираемый элемент равен искомому	- Да/Нет в зависимости от наличия элемента во множестве

ТЕОРЕТИКО-ЧИСЛОВЫЕ ЗАДАЧИ

Решаемая задача	Входные данные	Перебираемое множество	Проверяемое условие	Выход
Проверка числа на простоту	Целое число, не меньшее 2	Числа из диапазона от 2 до проверяемого числа	Деление на очередной перебираемый делитель	Да/Нет
Вычисление НОД	Два числа, для которых требуется вычислить НОД	Числа от 1 до минимального из входных чисел	Оба числа делятся на очередной перебираемый делитель	НОД
Вычисление НОК	Два числа, для которых требуется вычислить НОК	Числа от максимального до из произведения	Очередное перебираемое число делится на оба входных числа	НОК

ВЗЛОМ ШИФРОВ

Решаемая задача	Входные данные	Перебираемое множество	Проверяемое условие	Выходные данные
Взлом (поиск секретного ключа) шифра по известному шифрованному тексту	- Фрагмент шифрованного текста - Множество всех возможных секретных ключей - Перечень осмысленных слов и выражений (словарь)	Множество всех возможных ключей шифра	Пробное расшифрование шифрованного текста на очередном перебираемом ключе дает осмысленный (из словаря) открытый текст	Найденный секретный ключ , соответствующий условию перебора
Взлом (поиск секретного ключа) шифра по известному открытому тексту	- Фрагмент открытого текста - Соответствующий ему фрагмент шифрованного текста - Множество всех возможных секретных ключей	Множество всех возможных ключей шифра	Пробное шифрование открытого текста на очередном перебираемом ключе дает заданный шифрованный текст	Найденный секретный ключ , соответствующий условию перебора

ОПТИМИЗАЦИЯ ПОЛНОГО ПЕРЕБОРА ЗА СЧЕТ СОКРАЩЕНИЯ ПЕРЕБИРАЕМОГО МНОЖЕСТВА

Часто **перебираемое множество**, которое непосредственно вытекает из определения или постановки задачи, является **избыточным**. Его можно **сократить**, чтобы ускорить перебор или даже снизить его порядок

Определение. Число называется простым, если оно делится только на 1 и на само себя. Примеры простых чисел: **2, 3, 7, 13, 23**.

Минимальным простым числом является 2. Число 1 не является простым числом. Составное число - это число, у которого есть делители помимо 1 и самого себя.

Примеры составных чисел: 4, 9, 14, 100.

Если руководствоваться непосредственно определением, то перебирать нужно числа **от 2 до $N-1$**

ОПТИМИЗАЦИЯ ПЕРЕБОРА ПРИ ПРОВЕРКЕ ЧИСЛА НА ПРОСТОТУ

Все **четные** числа - составные за исключением 2.
Перебор можно свести только к нечетным числам и 2.
В результате перебор сократится **в 2 раза**.

Это изменит только константу, но **не поменяет порядок** сложности. Он останется $O(N)$, где N - проверяемое число.

Пример 1. Пусть $N = 24$, тогда $1 * 24 = 24$, $2 * 12 = 24$, $3 * 8 = 24$, $4 * 6 = 24$.
Как видно, один из делителей всегда **меньше $\sqrt{24}$**

Пример 2. **Точное равенство** достигается для полных квадратов.
Например, $25 = 5 * 5$. Здесь делители в точности равны $\sqrt{25}$

Таким образом, достаточно перебирать **до $\sqrt{N}$**
Это снижает степень сложность с **$O(N)$** до **$O(\sqrt{N})$**

ОБОСНОВАНИЕ КОРРЕКТНОСТИ АЛГОРИТМА

- **Заметим**, что для любого делителя P существует парный ему Q , такой что $P * Q = N$
- **Докажем**, что один из них всегда не превосходит $\sqrt{N}$
- Это легко доказать методом «от противного»
- Пусть оба $P > \sqrt{N}$ и $Q > \sqrt{N}$
- Тогда $P * Q > N$
- Однако по условию $P * Q = N$. Получаем противоречие $N > N$.
- **Пример 1.** Пусть $N = 24$, тогда $1 * 24 = 24$, $2 * 12 = 24$, $3 * 8 = 24$, $4 * 6 = 24$. Как видно, один из делителей всегда меньше $\sqrt{24}$
- **Пример 2.** Точное равенство достигается для полных квадратов. Например, $25 = 5 * 5$. Здесь делители в точности равны $\sqrt{25}$

Таким образом, действительно, достаточно перебирать **до $\sqrt{N}$**

ПРИМЕР СОВЕРШЕНСТВОВАНИЯ АЛГОРИТМА ВЫЧИСЛЕНИЯ НОД И НОК

Алгоритм Евклида основан на рекурсивном наблюдении, что
 $\text{НОД}(N, M) = \text{НОД}(N \bmod M, M)$
 $\text{НОД}(N, 0) = N$

Например, $\text{НОД}(93, 53) = \text{НОД}(53, 40) = \text{НОД}(40, 13) = \text{НОД}(13, 1) = 1$,
 $\text{НОД}(10000, 300) = \text{НОД}(300, 100) = \text{НОД}(100, 0)$.

Сложность алгоритма Евклида даже меньше, чем **логарифмическая**.

НОК связано с НОД соотношением $\text{НОК}(M, N) = M * N / \text{НОД}(M, N)$.

Поэтому алгоритм Евклида можно использовать для быстрого
вычисления НОК.

ЗАМЕР ВРЕМЕНИ

```
def isPrimeBasic(n):  
    for i in range(2, n-1):  
        if n % i == 0:  
            return False  
    return True
```

```
TRIALS = 1000  
time1 = time()  
for _ in range(TRIALS):  
    p = randint(100000, 1000000)  
    isPrimeBasic(p)  
time2 = time()  
print(time2 - time1)
```


ПРИМЕР: АЛГОРИТМЫ И КРИПТОГРАФИЯ

Законные пользователи: обладают **секретным ключом**, который позволяет им выполнять некоторые преобразования **«быстро»**

Незаконные пользователи: могут выполнить эти преобразования («взломать» защиту) с использованием настолько больших ресурсов («астрономических»), что **получить их на практике нереально**

Незаконные пользователи: могут «взломать» систему **с ничтожно малой вероятностью**.
Например, взять наугад секретный ключ, и он окажется верным

Какие операции?

- Зашифрование
- Расшифрование
- Подписание
- Генерация нужного числа
- Аутентификация

Для незаконного:

- Узнать секрет
- Узнать ключ

Упрощенно:

Законные пользователи
используют **полиномиальные**
алгоритмы

Незаконные пользователи
могут «взломать» только за
экспоненциальное время

ВОПРОСЫ ДЛЯ САМОКОНТРОЛЯ

1. Объясните понятие **алгоритма**, приведите примеры
2. Чем отличаются **детерминированные** и **вероятностные** алгоритмы?
3. Объясните понятие **псевдокода**, приведите примеры
4. Объясните понятие **блок-схемы**, приведите примеры
5. Как оценивается вычислительная **сложность** алгоритмов?
6. Виды функциональных **зависимостей**, характеризующих вычислительную сложность
7. Приведите примеры **элементарных** операций для различных алгоритмов
8. Объясните, почему некорректно и нецелесообразно напрямую **сравнивать** производительность последовательного и двоичного поиска
9. Что такое **практическая** сложность алгоритма, как и зачем она определяется?
10. Приведите примеры задач, которые можно решить полным **перебором**
11. Продемонстрируйте, как можно **оптимизировать** полный перебор на примере проверки числа на простоту
12. Объясните работу алгоритма **Евклида**

ПРАКТИЧЕСКОЕ ЗАДАНИЕ «ЗНАКОМСТВО»

СРОК – ДО 4 ОКТЯБРЯ

- Разработайте приложение с **произвольным UI** для работы со списком объектов. Объект характеризуется 4 свойствами (свойства перечислены в варианте)
- **Вместимость** не более $N_MAX = 12$ элементов. При попытке добавить (N_MAX+1) -ый **сообщить о невозможности** это сделать. При попытке удалить элемент из пустой базы тоже сообщить о невозможности это сделать
- При запуске должно содержаться **5 демо-объектов**, добавленных предварительно в программном коде (hard-code, скрипт и пр.)
- Приложение предполагает работу **меню из следующих пунктов**:
 - **Вывод** всех объектов в виде таблицы.
 - **Выход** из программы.
 - **Добавление** нового объекта (значения свойств вводит пользователь).
 - **Удаление** выбранного пользователем объекта (механизм выбора удаляемого элемента – на усмотрение студента).
 - **Сортировка** объектов по выбранному пользователем свойству (механизм выбора поля сортировки – на усмотрение студента).
 - **Фильтрация** (вывод объектов с указанным свойством) – согласно варианту.
 - **Преобразование** значения свойства у всех объектов – согласно варианту.

ПРАКТИЧЕСКОЕ ЗАДАНИЕ 1.1

СРОК – К СЛЕДУЮЩЕЙ ЛАБОРАТОРНОЙ

- Разработайте **схему экспериментов**, чтобы программа выводила наглядную **таблицу**, по которой можно видеть **зависимость** сложности от размера входных данных.
- Согласно разработанной схеме проведите экспериментальные **замеры времени** при различных входных данных для алгоритмов проверки числа на простоту:
 - Перебор от 2 до N
 - Перебор от 2 до N только четных чисел
 - Перебор от 2 до $\sqrt{N}$
 - Перебор от 2 до $\sqrt{N}$ только четных чисел
- Внедрите в реализации **переменные-счетчики** для определения практической сложности этих алгоритмов
- **Сопоставьте** все **три** полученных результата:
 - Замеры времени
 - Количество элементарных операций (практическую сложность)
 - Теоретическую сложность

ПРАКТИЧЕСКОЕ ЗАДАНИЕ 1.2

СРОК – К СЛЕДУЮЩЕЙ ЛАБОРАТОРНОЙ

- Согласно разработанной схеме проведите экспериментальные **замеры времени** при различных входных данных для алгоритмов вычисления НОД(M, N), в том числе перебором и алгоритмом Евклида:
 - Перебор от 1 до $\min(M, N)$
 - Перебор от $\min(M, N)$ до 1
 - Алгоритм Евклида
- Внедрите в реализации **переменные-счетчики** для определения практической сложности этих алгоритмов
- **Сопоставьте** все **три** полученных результата:
 - Замеры времени
 - Количество элементарных операций (практическую сложность)
 - Теоретическую сложность
- Выполните аналогичные задания для **НОК**

Спасибо за внимание !
